

1. Proje Özeti

Donanım Operasyon ve Servis; POS, tablet, yazıcı ve diğer donanımların kurulum, bakım, arıza ve teslim süreçlerini iş emri üzerinden yönetir. Operasyon yetkilisi işleri oluşturup teknisyenlere atar; teknisyen saha işlemlerini gerçekleştirir; süreç not, fotoğraf, GPS ve dijital imza ile kayıt altına alınır.

Platform: iOS 18+ (iPhone)

Dil: Swift 6 / SwiftUI

UI dili: Türkçe

Kod dili: İngilizce tanımlayıcılar

Mimari: Clean Architecture + MVVM + Repository + UseCase

Offline: SwiftData local-first + Sync Queue + Firebase

2. Roller ve Yetkiler

Üç sabit rol vardır. Yetki kaynağı RoleAccessPolicy (kod) + Firestore Security Rules.

- Admin: kullanıcı yönetimi, rol matrisi (salt okunur), sistem ayarları, sistem raporları. Saha işlemi ve düzenleme talebi onay/red YOK.
- Operasyon Yetkilisi: iş emri oluşturma/atama, tüm işleri görüntüleme, servis raporu, saha takibi, düzenleme talebi onay/red, sync çatışma çözümü.
- Teknisyen: atanan işler, durum akışı (kabul → yola çık → varış → başla → tamamla), not/fotoğraf/GPS/imza, düzenleme talebi oluşturma.

3. Katmanlı Mimari

Presentation → Domain → Data → Infrastructure. View'lar Firebase'e doğrudan bağlanmaz; ViewModel UseCase/Repository protokollerini kullanır; DIContainer bağımlılıkları enjekte eder.

Presentation

- DesignSystem: renk, tipografi, spacing, butonlar, StatusChip, WorkOrderCard vb.
- Admin / Operator / Technician ekranları ve typed NavigationStack router'ları.
- Auth, bildirimler, profil, rapor/PDF, saha takibi (harita + GPS).

Domain

- Entity'ler: User, Customer, WorkOrder, Note, Photo, Location, Signature, EditRequest, Notification...
- Policies: WorkOrderStateMachine, RoleAccessPolicy, CompletionRequirements, PhotoRequirements.
- UseCase'ler: Create/Assign/UpdateStatus/Complete, CaptureLocation/Signature, EditRequest approve/reject vb.

Data

- Local: SwiftData @Model + LocalPersistence + SwiftData*Repository.
- Remote: Firestore DTO/Repository + Firebase Storage (fotoğraf/imza).
- Sync: SyncManager, SyncQueue, SyncRemoteDispatcher, ConflictResolver (sunucu/yerel seçimi).

Infrastructure

- DIContainer (live/mock), NetworkReachability, LocationSampling / CoreLocation, FakeAuth (DEBUG).

4. İşlevsel Özellikler

İş Emri Yaşam Döngüsü

Atandı → Kabul Edildi → Yola Çıkıldı → Müşteriye Varıldı → İşlemde → Tamamlandı. Ek durum: Beklemede. Tamamlanan iş kilitlidir; yalnız onaylı EditRequest ile alan değişikliği.

Operasyon

- 6 adımlı Yeni İş Emri wizard'ı (tür, müşteri, cihaz, öncelik/tarih, teknisyen, özet).
- İş emri listesi/filtre, detay, teknisyen atama/yeniden atama.
- Düzenleme talebi gelen kutusu (onay/red).
- Saha Takibi: MapKit harita + GPS noktaları + yola çıkış/varış saatleri.
- Servis raporu (not/foto/GPS/imza) + PDF paylaşım.
- Senkron çatışma çözümü (sunucu/yerel).

Teknisyen

- Tek birincil CTA (duruma göre): Kabul Et / Yola Çık / Vardım / Başla / Tamamla / Devam Et.
- Yola çık / varış / tamamla anında GPS kaydı.
- Servis notları, fotoğraf (öncesi/sonrası/seri no), çift dijital imza.
- Tamamlama checklist'i (eksik alanlar açık gösterilir).
- Offline çalışma; internet gelince otomatik sync.

Admin

- Dashboard, kullanıcı CRUD/rol atama, rol izin matrisi (read-only), sistem ayarları.
- Sistem raporları ve iş emri bazlı detaylı rapor + PDF.

Ortak

- Firebase Auth giriş/çıkış, profil, şifre değiştirme, bildirim tercihleri.
- In-app bildirimler (operasyonel + sistem).
- DEBUG: DemoDataSeeder, mock GPS (simulator).

5. Teknolojiler ve Araçlar

Uygulama

- Swift 6.0, SwiftUI, Observation (@Observable), async/await, Sendable.
- SwiftData, CoreLocation, MapKit, AVFoundation / kamera, PhotosPicker.
- UIGraphicsPDFRenderer (servis raporu PDF).
- xcodegen (project.yml), SPM paket yönetimi.

Backend / Bulut

- Firebase Authentication (email/password).
- Cloud Firestore (yapısal veri + alt koleksiyonlar).
- Firebase Storage (fotoğraf ve imza binary).
- Firestore Security Rules (rol bazlı).

Mimari Kalıplar

- Clean Architecture, MVVM, Repository Pattern, UseCase, Constructor DI.
- Local-first yazma + Sync Queue; parent-before-child sync; pending:// Storage yolu.
- Role-based shell navigation (Admin / Operator / Technician).

Test ve Kalite

- XCTest (Domain, Data, Presentation regression suite'leri).

- DIContainer.mock + InMemory repository'ler.
- Hedef: iPhone / iOS 18+.

6. Veri Modeli (Özet)

- users, customers, workOrders
- workOrders/{id}/notes | statusHistory | photos | locations | signatures
- editRequests, notifications
- Storage: workOrders/{id}/photos/{photoId}, .../signatures/{signatureId}

7. Sync Akışı

UI/UseCase → Repository → SwiftData yaz + SyncOperation enqueue. NetworkMonitor tetikler → SyncManager sıralı drain → Firestore/Storage. Çatışmada ConflictResolver UI: Sunucu Verisini Kullan / Yerel Veriyi Kullan.

8. Bilinçli Kapsam Dışı (v1)

- Müşteri mobil uygulaması / müşteri login
- Ayrı cihaz envanter sistemi
- Operasyon dashboard genel haritası / takvim
- AI, chat, ödeme, puanlama, otomatik teknisyen atama
- Analytics / sosyal özellikler

9. Klasör Yapısı

App/ Presentation/ (DesignSystem, Admin, Operator, Technician, Navigation, Shared) Domain/ (Entities, Enums, UseCases, Policies, Repositories) Data/ (Local/SwiftData, Remote/Firebase, Sync) Infrastructure/DI Resources/ Tests/

Belge notu

Bu PDF, projenin mevcut mimarisini, kullanılan özellikleri ve teknolojileri özetler. Kaynak kod tek gerçek kaynaktır; yetki değişiklikleri RoleAccessPolicy ve Firestore rules ile yapılır.